Managing Agile Project Requirements with Storytest-Driven Development

Rick Mugridge, *Rimu Research*

Using FitLibrary eases storytest-driven development, which brings together requirements and automated testing ideas and practices to support agile software development.

In his 1997 book, *Managing the Design Factory*, Donald Reinhertsen argues,

> *In general, most product failures are caused by market risk. ... Managing market risk means capturing the requirements of what may be a diverse group of users. ... These people may have trouble expressing their needs, and their needs may be changing with time. This makes market risk a more formidable challenge than technical risk.*[1]

Agile software development approaches like Extreme Programming attempt to meet this market risk head-on.[2] These approaches accept that it can be nearly impossible to understand all the business requirements beforehand. XP principles and practices therefore support iterative development, with the whole team revising and refining the solution as they increase their understanding of market needs. Success depends on early and ongoing communication and feedback; developers ensure that their software is of high quality and thus amenable to change.

In XP, the *customer* role has domain expertise and assumes responsibility for directing the overall development from a business perspective. The customer uses high-level release planning to select the next release's work, which developers carry out in time-boxed iterations. Each business-level increment is called a *story*. Developers estimate each story's effort and the customer schedules a set of stories for the next iteration based on cost-benefit analysis.

In storytest-driven development, customers write *storytests*—executable, business-oriented examples—for each scheduled story.[3] The goal is to encourage clear communication of essential business needs (and ways of meeting those needs) using concrete examples. Often, writing storytests is a collaborative effort among team members and involves a range of skill sets. Because it's an evolutionary approach, it helps customers tease out what's valuable and possible. Furthermore, storytests evolve through collaboration and thus clarify the domain and scope for all project participants, enabling conversations that build shared understanding among team members. Finally, executing storytests as automated tests can help developers determine when new functionality is complete as well as if any existing functionality has been broken. Ward Cunningham's Fit framework uses tables to express storytests and is widely used for storytest development (http://fit.c2.com). FitNesse is a popular wiki-based tool that supports storytests, using Fit to run them as tests (www.fitnesse.org). I developed FitLibrary (http://sourceforge.net/projects/fitlibrary), which extends Fit to improve its business-level expressiveness. Here, I describe

 0740-7459/08/$25.00 © 2008 IEEE

storytest-driven development practices and offer storytest examples developed with FitLibrary to illustrate the process.

Moving beyond waterfall

It's not easy to develop nontrivial software that brilliantly meets the needs of an organization and its users. Too often, there's a mismatch between what's required and what's delivered, owing to misunderstandings about what is needed, possible, and valuable.[1]

In the early days, software developers assumed this mismatch was due largely to poor communication about what was needed. This led to the waterfall approach to software development, in which a project begins with business analysts gathering requirements from the system's prospective users. This in turn led to many new ideas, from interviewing techniques to systems and business analysis roles, to various notations—such as the Unified Modeling Language—to express requirements.

With the waterfall method, business analysts aim to understand all requirements prior to design and development. To achieve this, analysts pay attention to the following activities:

- A*nalyze relevant aspects of the business*. What are the business objects and their associations? What activities are carried out? What are the business rules?
- *Synthesize a design for a system embedded in the business*. What capabilities will the software system provide? How will the system support business processes, and how will that affect people's work?

Problems downstream

People have tried to answer all these questions in detail before starting software development using a mixture of written documents and notations. But the waterfall model has been less successful than anticipated due to several factors:[2,4]

- The requirements phase effectively freezes the scope, which encourages stakeholders to include lots of functionality that differs widely in its potential value to the business as a whole.
- Assessing the business value, impact, and interaction of specific system functions upfront is difficult because there are usually many unknowns.
- Understanding the essence of business processes is challenging because they're typically overlaid with potentially irrelevant details of how people did things in the past.
- Knowing when to stop writing requirements is often hard. What level of detail is appropriate? What do we expect readers to know?
- Translating intuitive business expertise into general and implementable statements about the business domain can be difficult. Many areas are not sufficiently formalized, which complicates automation.
- External and internal forces can lead to requirements changes during system development.
- As the project proceeds, understanding of the requirements and the solution evolves, and new opportunities can arise.
- The team might miss important requirements when stakeholders fail to identify and express their underlying assumptions.
- Programmers and testers must interpret the requirements documentation, and their understanding might be incomplete, inconsistent, or faulty without further person-to-person communication.

In addition, requirements documents tend to get outdated as things change. Business analysts tend not to refactor requirements as thinking changes about both the business needs and how the system will meet them. In addition, it's difficult to verify that documentation is consistent with the system (and vice versa); at best, teams derive separate tests from the written documentation and use them to test the resulting system.

The agile solution

According to agile approaches, understanding all the business requirements before development begins can be nearly impossible. Change is inevitable for several reasons, including that the larger business context changes. Also, as development proceeds, teams better understand three essential elements of the business and the solution:

- how things really work,
- what's possible, and
- the business value of specific functionality.

Given this, XP focuses on practices that encourage early and ongoing understanding of these issues and that allow the system to be revised and refined to meet that understanding. To enable this evolution, the requirements and the software need to evolve in tandem. Also, to remain pliable, code must be well designed. Finally, developers must be able to make changes with confidence. These needs led to the XP practice of test-driven development,[5] an innovation that sparked a host of fresh ideas,

Writing storytests helps customers tease out what's valuable and what's possible.

Applying Storytest-Driven Development

Many teams worldwide are using storytest-driven development to specify a wide range of application areas, from banking and broadcasting to embedded software in cell phones and automobiles.[1]

One team developing broadband services systems wrote storytests in Fit that provided extensive coverage and enabled the team to "reach a shared understanding of the domain and develop the ubiquitous language of the application."[2] In another case, domain experts wrote more than 300 storytests to support the porting of a legacy billing system.[3] Other developers used storytests to develop a transaction system for electronic data interchange. The system's customers and testers wrote 7,000 storytests, often in collaboration with the developers. The storytests helped the team "discover a lot of missing pieces or inconsistencies in a story." Overall, the team found that "executable acceptance testing is an effective tool for communicating, clarifying, and validating business requirements on a software project."[4]

References

1. U. Koch and M. Boxall, "Testing Mobile Handsets with FitNesse," 2006, http://video.google.co.uk/videoplay?docid=582823406455567909 (video).
2. P. Gandhi et al., "Creating a Living Specification Using Fit Documents," *Agile 2005 Conf.* (Agile 05), IEEE CS Press, 2005, pp. 253–258.
3. R. Bohnet and G. Meszaros, "Test-Driven Porting," *Agile 2005 Conf.* (Agile 05), IEEE CS Press, 2005, pp. 259–266.
4. G. Melnik and F. Maurer, "Multiple Perspectives on Executable Acceptance Test-Driven Development," *Proc. 8th Int'l Conf. Agile Processes in Software Eng. and Extreme Programming* (XP 07), LNCS 4536, Springer, 2007, pp. 245–249.

tools, and practices. In test-driven development, developers incrementally write programmer tests and use them to drive their thinking, design, development, and testing of the code that implements the system details. The code evolves over time, changing shape as the system grows and the team's understanding of it changes. Business-level tests support the stories developers use in iterative development; these "storytests" led to the emergence of storytest-driven development.[3,4]

Storytest-driven development

Storytest-driven development is a complementary form of test-driven development (TDD) applied to overall system development (see the sidebar "Applying Storytest-Driven Development" for examples).

Communicating business context and functionality

In storytest-driven development, customers write comprehensive storytests for each scheduled story in a system from the perspective of business-domain specialists, business analysts, product managers, and system testers. The resulting storytests help answer questions about the system in terms of its business context.

Storytests are an alternative to detailed requirements documents. They support ongoing conversations in the wider agile team. Rather than writing general statements about the business domain, customers use a set of examples to help clarify the general case. Also, instead of trying to cover the domain's full complexity from the start, customers evolve the storytest set over time. Finally, storytests are executable, so the team can continually verify their consistency with the system.

Storytests help developers drive the overall, iterative implementation of new functionality. They form one part of the ongoing communication between business- and development-oriented team members (who are preferably colocated). Developers take a storytest as the starting point for their next piece of work, using TDD when they drive changes into the application code. Questions that arise might lead customers to augment the storytests to cover cases they've missed or to clarify or refine the terminology.

Storytests and test-driven development

Storytest-driven development and TDD share several similarities. Both depend on advanced automated testing techniques as well as traditional testing approaches, including tests for nonfunctional requirements. With their common emphasis on evolution and emergent design, both approaches are ultimately more about designing than testing.

The two approaches differ in how they express executable examples. Developers usually write tests in the programming language they're using for development. In storytest-driven development, however, customers with business expertise—who are unlikely to know programming languages—write or review storytests. This fact has led to simpler notations for expressing storytests. The best known of these is found in the Fit framework tables (http://fit.c2.com).[4]

Expressing storytests with FitLibrary

With Fit, customers express storytests in tables and run them against the application software to check for compliance. Test results appear in the tables themselves, minimizing the cognitive dis-

tance between tests and results. FitLibrary (http://sourceforge.net/projects/fitlibrary) extends Fit to improve storytest expressiveness, encouraging a domain-driven design approach in which the examples directly express the domain objects and business-domain language.[6]

Following are a few simple FitLibrary storytests from the early stages of a payroll system's development. More examples are available in my book[4] as well as at www.fitnesse.org and http://sourceforge.net/projects/fitlibrary.

Workflow storytests

A simple story from early in a payroll system's iterative development is "An employee is paid a weekly wage based on the hours worked." Figure 1 shows a simple storytest for the action of paying an employee and the related tax. This storytest introduces much of the "syntax" of FitLibrary storytests.

Workflow storytests consist of an identification phase followed by three specification phases:

- *Within* identifies the target interaction context or domain object, which might be the overall application. In figure 1, for example, the target context is WageProcessing.
- *Given* specifies an assumed starting state as a set of property-value pairs. In figure 1, the properties are an employee and a related contract, and each has its own set of property-value pairs. For example, employee has employee id, name, and so on (the white cells on the right show the properties' values).
- *When* specifies a sequence of operations (workflow) in the given state's context. In figure 1, there's a single action with three arguments that contain employee details and weekly work hours. The nested table includes a collection of date-hours. The action's first cell—and every second one after that—holds a *keyword* (here, "pay employee," "with id," and "for hours worked").
- *Then* specifies the system's expected resulting state. This takes the same form as the Given phase, with property-value pairs. In figure 1, the employee and contact information remain unchanged, while a bank transfer and tax transfer result.

FitLibrary distinguishes cells that contain editable values from those containing labeling information. Action keywords help clarify the arguments' action and roles; along with the domain objects' property names, they form the domain's ubiquitous language.[6]

FitLibrary's conventions for defining states here are similar to those employed in a user interface—that is, it uses

- a pair of label and text fields to display or edit a single textual property and
- a table with a row for each item in the collection, with a header row giving each of the properties of those items.

It's important to stress that, although the storytests express the team's understanding of the business domain objects' essential nature, they in no way commit the team to actual user interface details.

When executing this storytest during testing, FitLibrary creates an object of the implementation class WageProcessing. FitLibrary then injects the Given phase table's data into this object to get it into the Given state. In testing terms, this phase does "setup." In keeping with domain-driven design principles, the Given phase's property names correspond directly to the underlying domain objects' properties. FitLibrary automatically converts names from tables into valid identifiers in the un-

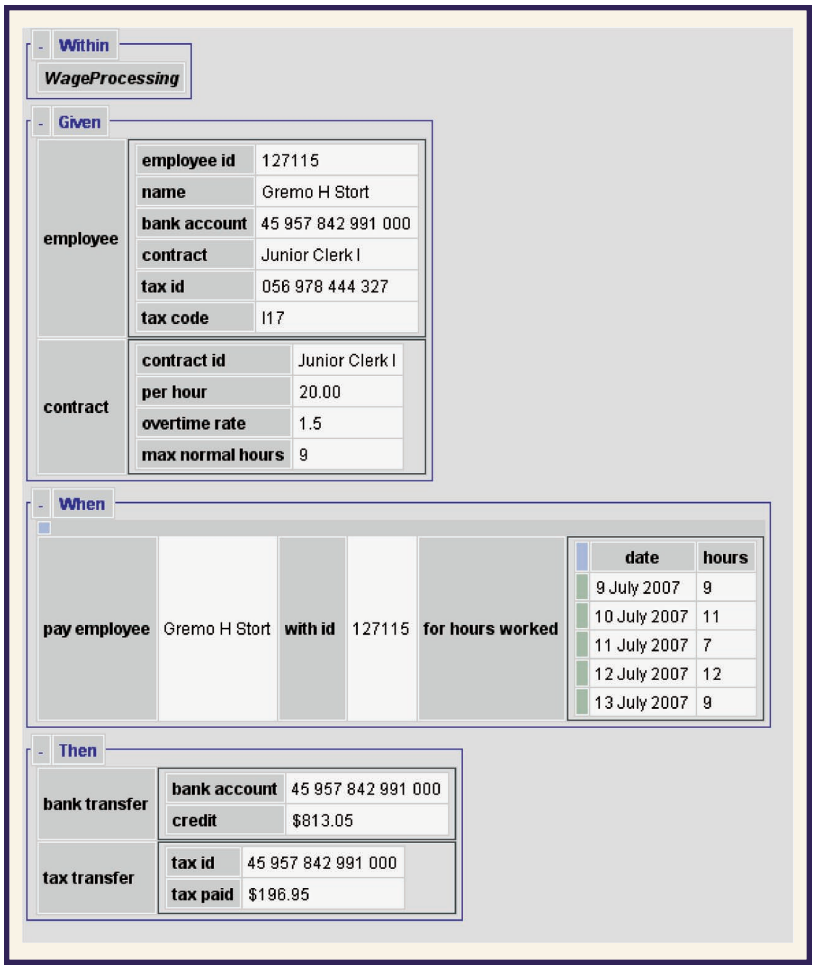


Figure 1. A simple workflow storytest describing the action of paying an employee, along with related taxes.

Figure 2. This example storytest, which shows how the organization determines ordinary and overtime hours, illustrates a calculation rule.

Within
WageProcessing

Calculate

hours	max ordinary	ordinary	overtime
9, 9, 9, 9, 9	9	45	0
9, 7	9	16	0
9, 0	9	9	0
0	9	0	0
11, 7	9	16	2
9, 9, 9, 9, 9	8	40	5
10,11,12,13,14	9	45	15

derlying programming language. As a result, FitLibrary needs almost no "fixturing" code to mediate between the tables and underlying domain code.

FitLibrary executes the When phase as method calls on the object under test. It automatically maps each action into a `WageProcessing` method. In this case, it calls the target object's `payEmployeeWithIdForHoursWorked()` method with three arguments. Because the method's declared third argument is a collection of `HoursWorked`, FitLibrary automatically creates a collection and fills it with `HoursWorked` objects for each row in the action's inner table.

In the Then phase, FitLibrary checks the actual value of each of the object's properties against those of the expected object. The report shows matching values in green and mismatches in red. FitLibrary reports any differences between the properties' expected and actual values in situ in the storytest tables.

Business-calculation storytests

To understand figure 1's workflow storytest, we must know how the organization determines overtime hours, applies rates, and calculates tax. An obvious approach—especially to those forced to test through the user interface—is to write a series of workflow storytests that cover all three underlying business rules and show their interactions. Unfortunately, this is difficult:

- We'd have to create several similar storytests and try to cover all the interesting cases. All this tedious copying and pasting can lead to mistakes.
- To understand the underlying business rule, we'd have to determine the differences among all the storytests, which obscures the rule.
- If a fundamental business rule changes, we'd have to review all the storytests and alter them and add new ones as needed.

Rather than adding more workflow stories to clarify the various underlying business rules, we can instead add focused *calculation* storytests for each one. Figure 2 shows a storytest for one of the underlying rules, illustrating how the organization determines `ordinary` and `overtime` hours, based on the hours worked and the contract's `max ordinary` number of hours.

Figure 2's storytest is structured differently than the workflow storytest in figure 1—each of figure 2's data rows is an example in its own right, with the given values to the left of the column of empty cells and the expected values to its right. The first row, for example, shows that there's no overtime when employees work nine hours each day. (I stripped away irrelevant data here—such as the particular employee—to keep the focus on the business rule's essence.) This makes it easier to identify missing cases or unaccounted for factors. If a team identifies two or more independent factors, they might further divide a calculation rule.

When FitLibrary executes a calculation storytest as a test, the report shows the right-hand column's cells in green or red, depending on whether the system's actual value is as expected or not. The column headers determine the methods that are called for each expected result. Each method might call directly into business-rule code in the target object, or it might carry out the workflow steps from figure 1, substituting data values from the current row.

With some applications, calculation storytests predominate. With others, workflow storytests are more significant. Often, the need for calculation and business-rule storytests isn't apparent until developers write some workflow storytests and consider related examples. Often, such rules haven't been previously formalized but rather are intuitively understood by business-domain experts.

Figure 3. An example of a rejected action. In this "Add a new contract" story, the contract is invalid because its overtime rate is too low.

Within
ContractConfiguration

Given

contracts

contract id	per hour	overtime rate	max normal hours
Junior Clerk I	20.00	1.5	9

When

reject add contract

contract id	Junior Clerk II
per hour	24.00
overtime rate	0.9
max normal hours	8

Many programmers would expect to write a unit test at this detailed level. However, this calculation rule is clearly in the business domain's realm and a domain expert must express it. Given this, domain-rich systems built using storytest-driven development tend to have fewer unit tests; those that remain typically focus on developing the technical infrastructure.

Sad-path workflow storytests

Figure 1's example covers a successful action, but we must also be able to specify a *sad-path* workflow storytest—that is, one that accounts for an action's failure, such as when a user violates business constraints. Consider a story to "Add a new contract." Figure 3 shows the example storytest and what happens when a user tries to add an invalid contract. The customer marks the When phase's single `add contract` action with `reject` to show that it won't succeed. The single argument is the new contract, which is invalid because the `overtime rate` isn't greater than 1.0.

`Reject` is a special FitLibrary annotation. When the application runs the storytest as a test, the `add contract` action should fail, returning a false Boolean value or throwing an exception. Other new contract constraints also apply, including `contract id` uniqueness. So, it's best here to write a single storytest with several concise examples that focus only on the target constraint—that is, a single business rule.

Business-constraint storytests

Figure 4's business-constraint storytest has five example rows, showing the `message` that's expected given the existing `contracts` and the `new contract`. The first row is for a valid `new contract`, with no error `message`. The second row fails a uniqueness constraint on the `contract id`. Subsequent rows cover constraints on the other new contract properties.

I assume that the storytest's table contains valid money amounts. I don't include a case where the `per hour` value is "lots," for example. Instead, independent constraint storytests define validation rules for money in general. Separation of concerns is just as important for storytests as it is for code.

In data-rich domains, constraints can involve an individual property value (such as "it must be positive"), groups of properties (such as "together they sum to 1.0"), as well as constraints over collections (such as "date ranges must not overlap"). When internationalization is applied to user messages, the storytest's expected `message` would be the message's internal form prior to local translation.

Developing storytests

Requirements are not always ripe for gathering.

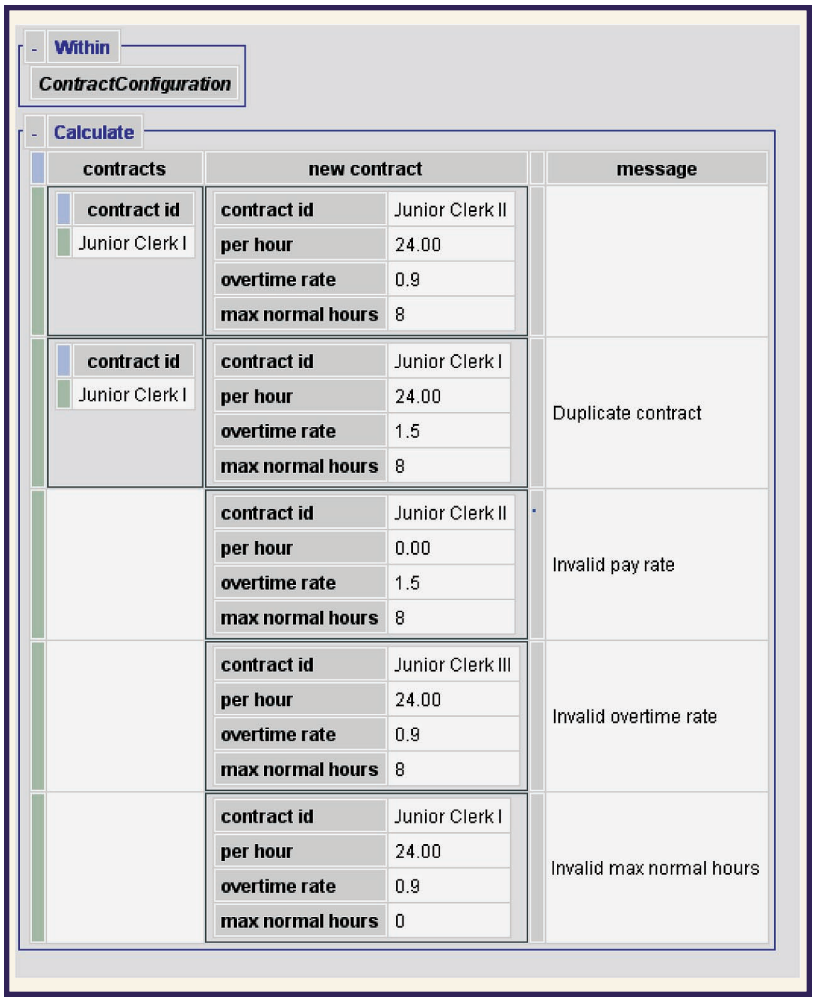


Figure 4. A business-constraint storytest. The second row shows the failure: the `contract id` violates a uniqueness constraint.

It's easy to imagine that concepts such as "kitchen," "fastener," and "hinge" have existed forever, but someone had to invent such helpful notions. It was only after regular use that they came to seem natural. Writing storytests often requires that we reconceptualize the domain, inventing new concepts that haven't been previously needed. It's not simply a matter of "writing it down" as the requirements-gathering notion assumes.

Practicing business experts apply their knowledge and background in their daily work. However, they might not have formalized their knowledge enough to make general statements that are valid and concise enough to define a system. Doing this requires abstraction skills.

Most of us have faced the awkwardness of listening to someone explain something complex using unfamiliar jargon, seemingly oblivious to our inability to understand them. Experts with a deep understanding of their domain are likely to find that it's easier to teach other people about it by starting simple, then gradually tackling more complexity. Examples can help considerably in this process.

Storytests are a business resource; to keep them clear and concise, we must regularly revise and reorganize them.

Getting started

Storytests draw on experts' facility with specifics by focusing their attention on concrete examples. In so doing, storytests act as a tool that can help experts clarify and communicate their knowledge. When writing storytests for functionality, I find it's best to have experts start with a simple example. Some experts find it difficult to remove interesting detail and create that first little example. However, doing so is a key step: it begins the process of selecting suitable terminology—the "ubiquitous language" in domain-driven design—and selecting suitable example values.

Once experts are happy with their first example, they can either expand it to include a little more complexity or create a new storytest. In this process, issues will arise, such as the need to make clear—and previously nonexistent—distinctions among different business objects.

The first few steps can be far from simple, even for someone experienced with both storytests and the domain. Concise, focused, and realistic examples work best, helping to educate the developers in the domain's specifics. Over several iterations, some storytests might change considerably as the writer better clarifies the business domain and moves beyond the simple cases.

When several experts work together to write storytests, creating concrete examples often highlights small differences of opinion that were previously unnoticed. Such teamwork works well when you need combined expertise to coherently assemble all system requirements.

Structuring storytests

To facilitate execution, storytest structure is constrained and requires a different skill than writing free text. To create an ever-growing set of system storytests, you need people with a range of skills and knowledge, including

- business knowledge;
- testing knowledge and skills (such as handling sad paths, boundary conditions, and odd cases that help clarify the domain);
- usability knowledge (such as knowledge of user tasks and user roles, which can help you choose stories and workflow storytests);
- object-oriented modeling and domain-driven design skills (for example, questions about a concrete domain object's structure and associations are likely to be helpful);
- abstraction skills and the ability to untangle and refactor storytests, particularly as understanding evolves and new business-domain concepts emerge; and
- the ability to ask the right questions (storytests often evolve in response to developers' questions during implementation, such as when code duplication might be due to missing storytest abstractions).

Storytests are a significant business resource; to keep them clear and concise, we must regularly revise and reorganize them. Processes such as design, refactoring, and design patterns also apply to storytests, as do design *smells*[4]—that is, indicators of poor quality. For example, a smelly storytest might

- have an unsuitable name and confusing tables that obscure its intent;
- use unrealistic examples that prevent the team from gaining a deep understanding of the domain;
- have a textual explanation that is better expressed in table form so that developers can automatically test it; and
- contain duplication, indicating the need to reconceptualize the domain or refactor the storytest (for example, you might extract a calculation rule from several workflow storytests).

Storytest-driven development helps bridge the gap between what organizations really need from a system and the system that developers actually deliver. This bridge builds on several factors. First, an iterative approach solves (or avoids) problems that arise from the inevitable speculation involved in upfront specification. Second, having storytests as executable documents drastically reduces the need to derive independent tests, reducing effort and the likelihood of translation errors. Third, storytests, as tests, help developers determine when added functionality is complete, and help ensure that it remains so. Finally, having storytests based on examples makes them widely useful as a thinking aid, as a focus for discussions and clarification, and as a general means of communication.

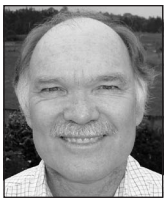
Storytest-driven development builds on and reuses ideas from previous requirements approaches, turning some of them "inside out." Storytests don't express class diagrams and use cases directly, for example, but they're likely to emerge; making such connections explicit is a fruitful area of future work. Another promising research area is to bring a stronger task-oriented (usability) perspective to storytests.

Storytest-driven development has other challenges as well:

- managing and organizing very large suites of storytests to make it easier to find those that are relevant to a particular persona, user task, interaction context, use case, or domain object, for example;
- keeping the storytests consistent with the underlying application code's structure and naming, especially when developers use a domain-driven design approach;
- assembling a team with all the needed skills to support high-quality storytest development; and
- finding the right abstraction level for storytests. Many people fall into the trap of writing storytests that explicitly test through the user interface. Such storytests tend to be verbose and fragile, are slow to run, and are unlikely to illuminate the domain.

Tool support is more advanced with test-driven development. Only now are tools becoming available that help domain experts keep track of the application's ubiquitous language and that support storytest evolution, including powerful refactoring support.

About the Author



Rick Mugridge runs Rimu Research, providing consulting and coaching in storytest-driven development and agile software development as well as consulting in software research and development. His research interests include agility, software tools, and general software development. He continues to develop FitLibrary, an open source storytesting tool, and is completing ZiBreve, a tool for refactoring storytests. He has a PhD in computer science from the University of Auckland, New Zealand, and is a member of the Agile Alliance. Contact him at 271 Ararimu Valley Rd., RD 2 Waimauku, Auckland 0882, New Zealand; rick@rimuresearch.com.

References

1. D.G. Reinertsen, *Managing the Design Factory*, The Free Press, 1997.
2. K. Beck, *eXtreme Programming Explained*, 2nd ed., Addison-Wesley, 2004.
3. J. Kerievsky, "Coined Storytest and Storytest Driven Development," *Industrial XP*, www.industrialxp.org/storyTdd.html.
4. R. Mugridge and W. Cunningham, *Fit for Developing Software*, Prentice Hall, 2005.
5. K. Beck, *Test Driven Development: By Example*, Addison-Wesley, 2002.
6. E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley, 2003.

IEEE computer society

PURPOSE: The IEEE Computer Society is the world's largest association of computing professionals and is the leading provider of technical information in the field.
MEMBERSHIP: Members receive the monthly magazine *Computer*, discounts, and opportunities to serve (all activities are led by volunteer members). Membership is open to all IEEE members, affiliate society members, and others interested in the computer field.
COMPUTER SOCIETY WEB SITE: www.computer.org
OMBUDSMAN: Email help@computer.org.

Next Board Meeting: 16 May 2008, Las Vegas, NV, USA

EXECUTIVE COMMITTEE

President: Michael R. Williams*
President-Elect: Rangachar Kasturi;* **Past President:** Deborah M. Cooper;* **VP, Conferences and Tutorials:** Susan K. (Kathy) Land (1ST VP);* **VP, Electronic Products and Services:** Sorel Reisman (2ND VP);* **VP, Chapters Activities:** Antonio Doria;* **VP, Educational Activities:** Stephen B. Seidman;† **VP, Publications:** Jon G. Rokne;† **VP, Standards Activities:** John Walz;† **VP, Technical Activities:** Stephanie M. White;* **Secretary:** Christina M. Schober;* **Treasurer:** Michel Israel;† **2006–2007 IEEE Division V Director:** Oscar N. Garcia;† **2007–2008 IEEE Division VIII Director:** Thomas W. Williams;† **2007 IEEE Division V Director-Elect:** Deborah M. Cooper;* ***Computer* Editor in Chief:** Carl K. Chang;† **Executive Director:** Angela R. Burgess†

* *voting member of the Board of Governors* † *nonvoting member of the Board of Governors*

BOARD OF GOVERNORS

Term Expiring 2007: Jean M. Bacon, George V. Cybenko, Antonio Doria, Richard A. Kemmerer, Itaru Mimura, Brian M. O'Connell, Christina M. Schober
Term Expiring 2008: Richard H. Eckhouse, James D. Isaak, James W. Moore, Gary McGraw, Robert H. Sloan, Makoto Takizawa, Stephanie M. White
Term Expiring 2009: Van L. Eden, Robert Dupuis, Frank E. Ferrante, Roger U. Fujii, Ann Q. Gates, Juan E. Gilbert, Don F. Shafer

EXECUTIVE STAFF

Executive Director: Angela R. Burgess; **Associate Executive Director:** Anne Marie Kelly; **Associate Publisher:** Dick Price; **Director, Administration:** Violet S. Doan; **Director, Finance and Accounting:** John Miller

COMPUTER SOCIETY OFFICES

Washington Office. 1730 Massachusetts Ave. NW, Washington, DC 20036-1992
Phone: +1 202 371 0101 • Fax: +1 202 728 9614 • Email: hq.ofc@computer.org
Los Alamitos Office. 10662 Los Vaqueros Circle, Los Alamitos, CA 90720-1314
Phone: +1 714 821 8380 • **Email: help@computer.org**
Membership and Publication Orders:
Phone: +1 800 272 6657 • Fax: +1 714 821 4641 • **Email: help@computer.org**
Asia/Pacific Office. Watanabe Building, 1-4-2 Minami-Aoyama, Minato-ku, Tokyo 107-0062, Japan
Phone: +81 3 3408 3118 • Fax: +81 3 3408 3553
Email: tokyo.ofc@computer.org

IEEE OFFICERS

President: Leah H. Jamieson; **President-Elect:** Lewis Terman; **Past President:** Michael R. Lightner; **Executive Director & COO:** Jeffry W. Raynes; **Secretary:** Celia Desmond; **Treasurer:** David Green; **VP, Educational Activities:** Moshe Kam; **VP, Publication Services and Products:** John Baillieul; **VP, Regional Activities:** Pedro Ray; **President, Standards Association:** George W. Arnold; **VP, Technical Activities:** Peter Staecker; **IEEE Division V Director:** Oscar N. Garcia; **IEEE Division VIII Director:** Thomas W. Williams; **President, IEEE-USA:** John W. Meredith, P.E.

revised 11 Oct. 2007

IEEE